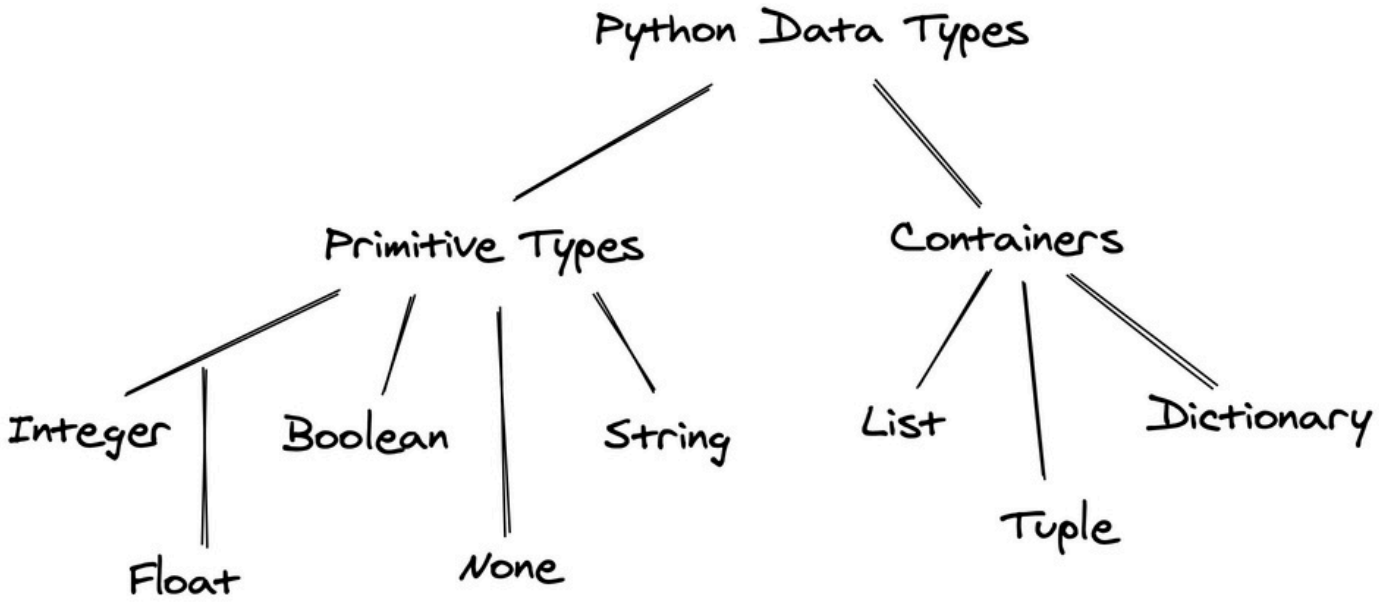
A Quick Tour of Variables and Data Types in Python



Part 2 of "Data Analysis with Python: Zero to Pandas"

This tutorial series is a beginner-friendly introduction to programming and data analysis using the Python programming language. These tutorials take a practical and coding-focused approach. The best way to learn the material is to execute the code and experiment with it yourself. Check out the full series here:

1. First Steps with Python and Jupyter
2. A Quick Tour of Variables and Data Types
3. Branching using Conditional Statements and Loops
4. Writing Reusable Code Using Functions
5. Reading from and Writing to Files
6. Numerical Computing with Python and Numpy
7. Analyzing Tabular Data using Pandas
8. Data Visualization using Matplotlib & Seaborn
9. Exploratory Data Analysis - A Case Study

This tutorial covers the following topics:

- Storing information using variables
- Primitive data types in Python: Integer, Float, Boolean, None and String
- Built-in data structures in Python: List, Tuple and Dictionary

- Methods and operators supported by built-in data types

How to run the code

This tutorial is an executable ~~Jupyter notebook~~ hosted on ~~Jovian~~. You can *run* this tutorial and experiment with the code examples in a couple of ways: *using free online resources* (recommended) or *on your computer*.

Option 1: Running using free online resources (1-click, recommended)

The easiest way to start executing the code is to click the **Run** button at the top of this page and select **Run on Binder**. You can also select "Run on Colab" or "Run on Kaggle", but you'll need to create an account on ~~Google Colab~~ or ~~Kaggle~~ to use these platforms.

Option 2: Running on your computer locally

To run the code on your computer locally, you'll need to set up ~~Python~~, download the notebook and install the required libraries. We recommend using the ~~Conda~~ distribution of Python. Click the **Run** button at the top of this page, select the **Run Locally** option, and follow the instructions.

> **Jupyter Notebooks**: This tutorial is a ~~Jupyter notebook~~ - a document made of *cells*. Each cell can contain code written in Python or explanations in plain English. You can execute code cells and view the results, e.g., numbers, messages, graphs, tables, files, etc. instantly within the notebook. Jupyter is a powerful platform for experimentation and analysis. Don't be afraid to mess around with the code & break things - you'll learn a lot by encountering and fixing errors. You can use the "Kernel > Restart & Clear Output" menu option to clear all outputs and start again from the top.

Storing information using variables

Computers are useful for two purposes: storing information (also known as data) and performing operations on stored data. While working with a programming language such as Python, data is stored in variables. You can think of variables are containers for storing data. The data stored within a variable is called its value. Creating variables in Python is pretty easy, as we've already seen in the previous tutorial.

```
my_favorite_color = "blue"
```

```
my_favorite_color
```

```
'blue'
```

A variable is created using an assignment statement. It begins with the variable's name, followed by the assignment operator = followed by the value to be stored within the variable. Note that the assignment operator = is different from the equality comparison operator ==.

You can also assign values to multiple variables in a single statement by separating the variable names and values with commas.

```
color1, color2, color3 = "red", "green", "blue"
```

```
color1
```

```
'red'
```

```
color2
```

```
'green'
```

```
color3
```

```
'blue'
```

You can assign the same value to multiple variables by chaining multiple assignment operations within a single statement.

```
color4 = color5 = color6 = "magenta"
```

```
color4
```

```
'magenta'
```

```
color5
```

```
'magenta'
```

```
color6
```

```
'magenta'
```

You can change the value stored within a variable by assigning a new value to it using another assignment statement. Be careful while reassigning variables: when you assign a new value to the variable, the old value is lost and no longer accessible.

```
my_favorite_color = "red"
```

```
my_favorite_color
```

```
'red'
```

While reassigning a variable, you can also use the variable's previous value to compute the new value.

```
counter = 10
```

```
counter = counter + 1
```

```
counter
```

```
11
```

The pattern `var = var op something` (where `op` is an arithmetic operator like `+`, `-`, `*`, `/`) is very common, so Python provides a *shorthand* syntax for it.

```
counter = 10
```

```
# Same as `counter = counter + 4`
counter += 4
```

```
counter
```

```
14
```

Variable names can be short (`a`, `x`, `y`, etc.) or descriptive (`my_favorite_color`, `profit_margin`, `the_3_musketeers`, etc.). However, you must follow these rules while naming Python variables:

- A variable's name must start with a letter or the underscore character `_`. It cannot begin with a number.
- A variable name can only contain lowercase (small) or uppercase (capital) letters, digits, or underscores (`a-z`, `A-Z`, `0-9`, and `_`).
- Variable names are case-sensitive, i.e., `a_variable`, `A_Variable`, and `A_VARIABLE` are all different variables.

Here are some valid variable names:

```
a_variable = 23
is_today_Saturday = False
my_favorite_car = "Delorean"
the_3_musketeers = ["Athos", "Porthos", "Aramis"]
```

Let's try creating some variables with invalid names. Python prints a syntax error if your variable's name is invalid.

> **Syntax**: The syntax of a programming language refers to the rules that govern the structure of a valid instruction or *statement*. If a statement does not follow these rules, Python stops execution and informs you that there is a *syntax error*. You can think of syntax as the rules of grammar for a programming language.

```
a variable = 23
```

```
  File "<ipython-input-20-b4f9daa715a5>", line 1
    a variable = 23
      ^
SyntaxError: invalid syntax
```

```
is_today_$aturday = False
```

```
  File "<ipython-input-21-3785425ca227>", line 1
    is_today_$aturday = False
                ^
SyntaxError: invalid syntax
```

```
my-favorite-car = "Delorean"
```

```
  File "<ipython-input-22-829f5ed5b10b>", line 1
    my-favorite-car = "Delorean"
    ^
SyntaxError: cannot assign to operator
```

```
3_musketeers = ["Athos", "Porthos", "Aramis"]
```

```
  File "<ipython-input-23-41d8638a15bb>", line 1
    3_musketeers = ["Athos", "Porthos", "Aramis"]
     ^
SyntaxError: invalid decimal literal
```

Save and upload your notebook

Whether you're running this Jupyter notebook online or on your computer, it's essential to save your work from time to time. You can continue working on a saved notebook later or share it with friends and colleagues to let them execute your code. Jovian offers an easy way of saving and sharing your Jupyter notebooks online.

```
# Install the jovian library
!pip install jovian --upgrade --quiet
```

```
import jovian
```

```
jovian.commit(project='python-variables-and-data-types')
```

```
[jovian] Attempting to save notebook..
[jovian] Updating notebook "aakashns/python-variables-and-data-types" on https:/
[jovian] Uploading notebook..
[jovian] Capturing environment..
[jovian] Committed successfully! https://jovian.ai/aakashns/python-variables-and
'https://jovian.ai/aakashns/python-variables-and-data-types'
```

The first time you run `jovian.commit`, you'll be asked to provide an API Key to securely upload the notebook to your Jovian account. You can get the API key from your Jovian profile page after logging in / signing up.

`jovian.commit` uploads the notebook to your Jovian account, captures the Python environment, and creates a shareable link for your notebook, as shown above. You can use this link to share your work and let anyone (including you) run your notebooks and reproduce your work.

Built-in data types in Python

Any data or information stored within a Python variable has a *type*. You can view the type of data stored within a variable using the `type` function.

```
a_variable
```

```
23
```

```
type(a_variable)
```

```
int
```

```
is_today_Saturday
```

```
False
```

```
type(is_today_Saturday)
```

```
bool
```

```
my_favorite_car
```

```
'Delorean'
```

```
type(my_favorite_car)
```

```
str
```

```
the_3_musketeers
```

```
['Athos', 'Porthos', 'Aramis']
```

```
type(the_3_musketeers)
```

```
list
```

Python has several built-in data types for storing different kinds of information in variables. Following are some commonly used data types:

1. Integer
2. Float
3. Boolean
4. None
5. String
6. List
7. Tuple
8. Dictionary

Integer, float, boolean, None, and string are *primitive data types* because they represent a single value. Other data types like list, tuple, and dictionary are often called *data structures* or *containers* because they hold multiple pieces of data together.

Integer

Integers represent positive or negative whole numbers, from negative infinity to infinity. Note that integers should not include decimal points. Integers have the type `int`.

```
current_year = 2020
```

```
current_year
```

```
2020
```

```
type(current_year)
```

```
int
```

Unlike some other programming languages, integers in Python can be arbitrarily large (or small). There's no lowest or highest value for integers, and there's just one `int` type (as opposed to `short, int, long, long long, unsigned int, etc. in C/C++/Java`).

```
a_large_negative_number = -233740383748329343342343173483 43
```

```
a_large_negative_number
```

```
-233740383748329343342343173483 43
```

```
type(a_large_negative_number)
```

```
int
```

Float

Floats (or floating-point numbers) are numbers with a decimal point. There are no limits on the value or the number of digits before or after the decimal point. Floating-point numbers have the type `float` .

```
pi = 3.141592653589793238
```

```
pi
```

```
3.141592653589793
```

```
type(pi)
```

```
float
```

Note that a whole number is treated as a float if written with a decimal point, even though the decimal portion of the number is zero.

```
a_number = 3.0
```

```
a_number
```

```
3.0
```

```
type(a_number)
```

```
float
```

```
another_number = 4.
```

```
another_number
```

```
4.0
```

```
type(another_number)
```

```
float
```

Floating point numbers can also be written using the scientific notation with an "e" to indicate the power of 10.

```
one_hundredth = 1e-2
```

```
one_hundredth
```

```
0.01
```

```
type(one_hundredth)
```

```
float
```

```
avogadro_number = 6.02214076e23
```

```
avogadro_number
```

```
6.02214076e+23
```

```
type(avogadro_number)
```

```
float
```

You can convert floats into integers and vice versa using the `float` and `int` functions. The operation of converting one type of value into another is called casting.

```
float(current_year)
```

```
2020.0
```

```
float(a_large_negative_number)
```

```
-2.3374038374832935e+31
```

```
int(pi)
```

```
3
```

```
int(avogadro_number)
```

```
602214075999999987023872
```

While performing arithmetic operations, integers are automatically converted to `floats` if any of the operands is a `float`. Also, the division operator `/` always returns a float, even if both operands are integers. Use the `//` operator if you want the result of the division to be an int.

```
type(45 * 3.0)
```

```
float
```

```
type(45 * 3)
```

```
int
```

```
type(10/3)
```

```
float
```

```
type(10/2)
```

```
float
```

```
type(10//2)
```

```
int
```

Boolean

Booleans represent one of 2 values: `True` and `False`. Booleans have the type `bool`.

```
is_today_Sunday = True
```

```
is_today_Sunday
```

```
True
```

```
type(is_today_Saturday)
```

```
bool
```

Booleans are generally the result of a comparison operation, e.g., `==`, `>=`, etc.

```
cost_of_ice_bag = 1.25
is_ice_bag_expensive = cost_of_ice_bag >= 10
```

```
is_ice_bag_expensive
```

```
False
```

```
type(is_ice_bag_expensive)
```

```
bool
```

Booleans are automatically converted to `int`s when used in arithmetic operations. `True` is converted to `1` and False is converted to `0`.

```
5 + False
```

```
5
```

```
3. + True
```

```
4.0
```

Any value in Python can be converted to a Boolean using the `bool` function.

Only the following values evaluate to `False` (they are often called *falsy* values):

1. The value `False` itself
2. The integer `0`
3. The float `0.0`
4. The empty value `None`
5. The empty text `""`
6. The empty list `[]`
7. The empty tuple `()`
8. The empty dictionary `{}`
9. The empty set `set()`
10. The empty range `range(0)`

Everything else evaluates to `True` (a value that evaluates to True is often called a *truthy* value).

```
bool(False)
```

```
False
```

```
bool(0)
```

```
False
```

```
bool(0.0)
```

```
False
```

```
bool(None)
```

```
False
```

```
bool("")
```

```
False
```

```
bool([])
```

```
False
```

```
bool(())
```

```
False
```

```
bool({})
```

```
False
```

```
bool(set())
```

```
False
```

```
bool(range(0))
```

```
False
```

```
bool(True), bool(1), bool(2.0), bool("hello"), bool([1,2]), bool((2,3)), bool(range(
```

```
(True, True, True, True, True, True, True)
```

None

The None type includes a single value `None`, used to indicate the absence of a value. None has the type `NoneType`. It is often used to declare a variable whose value may be assigned later.

```
nothing = None
```

```
type(nothing)
```

```
NoneType
```

String

A string is used to represent text (*astring ofcharacters*) in Python. Strings must be surrounded using quotations (either the single quote `'` or the double quote `"`). Strings have the type `string`.

```
today = "Saturday"
```

```
today
```

```
'Saturday'
```

```
type(today)
```

```
str
```

You can use single quotes inside a string written with double quotes, and vice versa.

```
my_favorite_movie = "One Flew over the Cuckoo's Nest"
```

```
my_favorite_movie
```

```
"One Flew over the Cuckoo's Nest"
```

```
my_favorite_pun = 'Thanks for explaining the word "many" to me, it means a lot.'
```

```
my_favorite_pun
```

```
'Thanks for explaining the word "many" to me, it means a lot.'
```

To use a double quote within a string written with double quotes, *escape* the inner quotes by prefixing them with the `\` character.

```
another_pun = "The first time I got a universal remote control, I thought to myself
```

```
another_pun
```

```
'The first time I got a universal remote control, I thought to myself "This
changes everything".'
```

Strings created using single or double quotes must begin and end on the same line. To create multiline strings, use three single quotes `'''` or three double quotes `"""` to begin and end the string. Line breaks are represented using the newline character `\n`.

```
yet_another_pun = '''Son: "Dad, can you tell me what a solar eclipse is?"
Dad: "No sun."'''
```

```
yet_another_pun
```

```
'Son: "Dad, can you tell me what a solar eclipse is?" \nDad: "No sun."'
```

Multiline strings are best displayed using the `print` function.

```
print(yet_another_pun)
```

```
Son: "Dad, can you tell me what a solar eclipse is?"
Dad: "No sun."
```

```
a_music_pun = """
Two windmills are standing in a field and one asks the other,
"What kind of music do you like?"

The other says,
"I'm a big metal fan."
"""


print(a_music_pun)
```

```
Two windmills are standing in a field and one asks the other,
"What kind of music do you like?"

The other says,
"I'm a big metal fan."
```

You can check the length of a string using the `len` function.

```
len(my_favorite_movie)
```

```
31
```

Note that special characters like `\n` and escaped characters like `\"` count as a single character, even though they are written and sometimes printed as two characters.

```
multiline_string = """a
b"""
multiline_string
```

```
'a\nb'
```

```
len(multiline_string)
```

```
3
```

A string can be converted into a list of characters using `list` function.

```
list(multiline_string)
```

```
['a', '\n', 'b']
```

Strings also support several list operations, which are discussed in the next section. We'll look at a couple of examples here.

You can access individual characters within a string using the `[]` indexing notation. Note the character indices go from `0` to n-1, where n is the length of the string.

```
today = "Saturday"
```

```
today[0]
```

```
'S'
```

```
today[3]
```

```
'u'
```

```
today[7]
```

```
'y'
```

You can access a part of a string using by providing a `start:end` range instead of a single index in `[]`.

```
today[5:8]
```

```
'day'
```

You can also check whether a string contains a some text using the `in` operator.

```
'day' in today
```

```
True
```

```
'Sun' in today
```

```
False
```

Two or more strings can be joined or *concatenated* using the `+` operator. Be careful while concatenating strings, sometimes you may need to add a space character `" "` between words.

```
full_name = "Derek O'Brien"
```

```
greeting = "Hello"
```

```
greeting + full_name
```

```
"HelloDerek O'Brien"
```

```
greeting + " " + full_name + "!" # additional space
```

```
"Hello Derek O'Brien!"
```

Strings in Python have many built-in *methods* that are used to manipulate them. Let's try out some common string methods.

> **Methods**: Methods are functions associated with data types and are accessed using the dot notation e.g. variable_name.method() or "a string".method().
> Methods are a powerful technique for associating common operations with values of specific data types.

The .lower(), .upper() and .capitalize() methods are used to change the case of the characters.

```
today.lower()
```

```
'saturday'
```

```
"saturday".upper()
```

```
'SATURDAY'
```

```
"monday".capitalize() # changes first character to uppercase
```

```
'Monday'
```

The .replace method replaces a part of the string with another string. It takes the portion to be replaced and the replacement text as *inputs* or *arguments*.

```
another_day = today.replace("Satur", "Wednes")
```

```
another_day
```

```
'Wednesday'
```

Note that `replace` returns a new string, and the original string is not modified.

```
today
```

```
'Saturday'
```

The `.split` method splits a string into a list of strings at every occurrence of provided character(s).

```
"Sun,Mon,Tue,Wed,Thu,Fri,Sat".split(",")
```

```
['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat']
```

The `.strip` method removes whitespace characters from the beginning and end of a string.

```
a_long_line = "      This is a long line with some space before, after,     and som
```

```
a_long_line_stripped = a_long_line.strip()
```

```
a_long_line_stripped
```

```
'This is a long line with some space before, after,                    and some space in the middle..'
```

The `.format` method combines values of other data types, e.g., integers, floats, booleans, lists, etc. with strings. You can use `format` to construct output messages for display.

```
# Input variables
cost_of_ice_bag = 1.25
profit_margin = .2
number_of_bags = 500

# Template for output message
output_template = """If a grocery store sells ice bags at $ {} per bag, with a profi
then the total profit it makes by selling {} ice bags is $ {}."""

print(output_template)
```

```
If a grocery store sells ice bags at $ {} per bag, with a profit margin of {} %,
then the total profit it makes by selling {} ice bags is $ {}.
```

```
# Inserting values into the string
total_profit = cost_of_ice_bag * profit_margin * number_of_bags
```

```
output_message = output_template.format(cost_of_ice_bag, profit_margin*100, number_o

print(output_message)
```

```
If a grocery store sells ice bags at $ 1.25 per bag, with a profit margin of 20.
then the total profit it makes by selling 500 ice bags is $ 125.0.
```

Notice how the placeholders {} in the output_template string are replaced with the arguments provided to the .format method.

It is also possible to use the string concatenation operator + to combine strings with other values. However, those values must first be converted to strings using the str function.

```
"If a grocery store sells ice bags at $ " + cost_of_ice_bag + ", with a profit margi
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-127-78b7958ec7cc> in <module>
----> 1 "If a grocery store sells ice bags at $ " + cost_of_ice_bag + ", witha
profit margin of " + profit_margin

TypeError: can only concatenate str (not "float") to str
```

```
"If a grocery store sells ice bags at $ " + str(cost_of_ice_bag) + ", with a profit
```

```
'If a grocery store sells ice bags at $ 1.25, with a profit margin of 0.2'
```

You can str to convert a value of any data type into a string.

```
str(23)
```

```
'23'
```

```
str(23.432)
```

```
'23.432'
```

```
str(True)
```

```
'True'
```

```
the_3_musketeers = ["Athos", "Porthos", "Aramis"]
str(the_3_musketeers)
```

```
"['Athos', 'Porthos', 'Aramis']"
```

Note that all string methods return new values and DO NOT change the existing string. You can find a full list of string methods here: https://www.w3schools.com/python/python_ref_string.asp.

Strings also support the comparison operators `==` and `!=` for checking whether two strings are equal.

```
first_name = "John"
```

```
first_name == "Doe"
```

```
False
```

```
first_name == "John"
```

```
True
```

```
first_name != "Jane"
```

```
True
```

We've looked at the primitive data types in Python. We're now ready to explore non-primitive data structures, also known as containers.

Before continuing, let us run `jovian.commit` once again to record another snapshot of our notebook.

```
jovian.commit()
```

```
[jovian] Attempting to save notebook..
[jovian] Updating notebook "aakashns/python-variables-and-data-types" on https:/
[jovian] Uploading notebook..
[jovian] Capturing environment..
[jovian] Committed successfully! https://jovian.ai/aakashns/python-variables-and
'https://jovian.ai/aakashns/python-variables-and-data-types'
```

Running `jovian.commit` multiple times within a notebook records new versions automatically. You will continue to have access to all the previous versions of your notebook, using the versions dropdown on the notebook's Jovian.

List

A list in Python is an ordered collection of values. Lists can hold values of different data types and support operations to add, remove, and change values. Lists have the type `list`.

To create a list, enclose a sequence of values within square brackets `[` and `]`, separated by commas.

```
fruits = ['apple', 'banana', 'cherry']
```

```
fruits
```

```
['apple', 'banana', 'cherry']
```

```
type(fruits)
```

```
list
```

Let's try creating a list containing values of different data types, including another list.

```
a_list = [23, 'hello', None, 3.14, fruits, 3 <= 5]
```

```
a_list
```

```
[23, 'hello', None, 3.14, ['apple', 'banana', 'cherry'], True]
```

```
empty_list = []
```

```
empty_list
```

```
[]
```

To determine the number of values in a list, use the `len` function. You can use `len` to determine the number of values in several other data types.

```
len(fruits)
```

```
3
```

```
print("Number of fruits:", len(fruits))
```

```
Number of fruits: 3
```

```
len(a_list)
```

6

```
len(empty_list)
```

0

You can access an element from the list using its *index*, e.g., `fruits[2]` returns the element at index 2 within the list `fruits`. The starting index of a list is 0.

```
fruits[0]
```

'apple'

```
fruits[1]
```

'banana'

```
fruits[2]
```

'cherry'

If you try to access an index equal to or higher than the length of the list, Python returns an `IndexError`.

```
fruits[3]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-152-7ceeafd384d7> in <module>
----> 1 fruits[3]

IndexError: list index out of range
```

```
fruits[4]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-153-b8c91da6ba3a> in <module>
----> 1 fruits[4]

IndexError: list index out of range
```

You can use negative indices to access elements from the end of a list, e.g., `fruits[-1]` returns the last element, `fruits[-2]` returns the second last element, and so on.

```
fruits[-1]
```

```
'cherry'
```

```
fruits[-2]
```

```
'banana'
```

```
fruits[-3]
```

```
'apple'
```

```
fruits[-4]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
<ipython-input-157-1cb2d66442ee> in <module>
----> 1 fruits[-4]

IndexError: list index out of range
```

You can also access a range of values from the list. The result is itself a list. Let us look at some examples.

```
a_list = [23, 'hello', None, 3.14, fruits, 3 <= 5]
```

```
a_list
```

```
[23, 'hello', None, 3.14, ['apple', 'banana', 'cherry'], True]
```

```
len(a_list)
```

```
6
```

```
a_list[2:5]
```

```
[None, 3.14, ['apple', 'banana', 'cherry']]
```

Note that the range `2:5` includes the element at the start index 2 but does not include the element at the end index `5`. So, the result has 3 values (index 2, 3, and 4).

Here are some experiments you should try out (use the empty cells below):

- Try setting one or both indices of the range are larger than the size of the list, e.g., `a_list[2:10]`
- Try setting the start index of the range to be larger than the end index, e.g., `list_a[2:10]`
- Try leaving out the start or end index of a range, e.g., `a_list[2:]` or `a_list[:5]`
- Try using negative indices for the range, e.g., `a_list[-2:-5]` or `a_list[-5:-2]` (can you explain the results?)

> The flexible and interactive nature of Jupyter notebooks makes them an excellent tool for learning and experimentation. If you are new to Python, you can resolve most questions as soon as they arise simply by typing the code into a cell and executing it. Let your curiosity run wild, discover what Python is capable of and what it isn't!

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

You can also change the value at a specific index within a list using the assignment operation.

```
fruits
```

```
['apple', 'banana', 'cherry']
```

```
fruits[1] = 'blueberry'
```

```
fruits
```

```
['apple', 'blueberry', 'cherry']
```

A new value can be added to the end of a list using the `append` method.

```
fruits.append('dates')
```

```
fruits
```

```
['apple', 'blueberry', 'cherry', 'dates']
```

A new value can also be inserted at a specific index using the `insert` method.

```
fruits.insert(1, 'banana')
```

```
fruits
```

```
['apple', 'banana', 'blueberry', 'cherry', 'dates']
```

You can remove a value from a list using the `remove`method.

```
fruits.remove('blueberry')
```

```
fruits
```

```
['apple', 'banana', 'cherry', 'dates']
```

What happens if a list has multiple instances of the value passed to `.remove`? Try it out.

Start coding or generate with AI.

Start coding or generate with AI.

To remove an element from a specific index, use the `pop` method. The method also returns the removed element.

```
fruits
```

```
['apple', 'banana', 'cherry', 'dates']
```

```
fruits.pop(1)
```

```
'banana'
```

```
fruits
```

```
['apple', 'cherry', 'dates']
```

If no index is provided, the `pop` method removes the last element of the list.

```
fruits.pop()
```

```
'dates'
```

```
fruits
```

```
['apple', 'cherry']
```

You can test whether a list contains a value using the `in` operator.

```
'pineapple' in fruits
```

```
False
```

```
'cherry' in fruits
```

```
True
```

To combine two or more lists, use the `+` operator. This operation is also called *concatenation*.

```
fruits
```

```
['apple', 'cherry']
```

```
more_fruits = fruits + ['pineapple', 'tomato', 'guava'] + ['dates', 'banana']
```

```
more_fruits
```

```
['apple', 'cherry', 'pineapple', 'tomato', 'guava', 'dates', 'banana']
```

To create a copy of a list, use the `copy` method. Modifying the copied list does not affect the original.

```
more_fruits_copy = more_fruits.copy()
```

```
more_fruits_copy
```

```
['apple', 'cherry', 'pineapple', 'tomato', 'guava', 'dates', 'banana']
```

```
# Modify the copy
more_fruits_copy.remove('pineapple')
more_fruits_copy.pop()
more_fruits_copy
```

```
['apple', 'cherry', 'tomato', 'guava', 'dates']
```

```
# Original list remains unchanged
more_fruits
```

```
['apple', 'cherry', 'pineapple', 'tomato', 'guava', 'dates', 'banana']
```

Note that you cannot create a copy of a list by simply creating a new variable using the assignment operator =. The new variable will point to the same list, and any modifications performed using either variable will affect the other.

```
more_fruits
```

```
['apple', 'cherry', 'pineapple', 'tomato', 'guava', 'dates', 'banana']
```

```
more_fruits_not_a_copy = more_fruits
```

```
more_fruits_not_a_copy.remove('pineapple')
more_fruits_not_a_copy.pop()
```

```
'banana'
```

```
more_fruits_not_a_copy
```

```
['apple', 'cherry', 'tomato', 'guava', 'dates']
```

```
more_fruits
```

```
['apple', 'cherry', 'tomato', 'guava', 'dates']
```

Just like strings, there are several in-built methods to manipulate a list. However, unlike strings, most list methods modify the original list rather than returning a new one. Check out some common list operations here: https://www.w3schools.com/python/python_ref_list.asp .

Following are some exercises you can try out with list methods (use the blank code cells below):

- Reverse the order of elements in a list
- Add the elements of one list at the end of another list
- Sort a list of strings in alphabetical order

- Sort a list of numbers in decreasing order

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Tuple

A tuple is an ordered collection of values, similar to a list. However, it is not possible to add, remove, or modify values in a tuple. A tuple is created by enclosing values within parentheses `(` and `)`, separated by commas.

> Any data structure that cannot be modified after creation is called *immutable*. You can think of tuples as immutable lists.

Let's try some experiments with tuples.

```
fruits = ('apple', 'cherry', 'dates')
```

```
# check no. of elements
len(fruits)
```

```
3
```

```
# get an element (positive index)
fruits[0]
```

```
'apple'
```

```
# get an element (negative index)
fruits[-2]
```

```
'cherry'
```

```
# check if it contains an element
'dates' in fruits
```

```
True
```

```
# try to change an element
fruits[0] = 'avocado'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-195-eea1d48cc8b5> in <module>
      1 # try to change an element
----> 2 fruits[0] = 'avocado'

TypeError: 'tuple' object does not support item assignment
```

```
# try to append an element
fruits.append('blueberry')
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-196-e5ea20adaaf8> in <module>
1 # try to append an element
----> 2 fruits.append('blueberry')

AttributeError: 'tuple' object has no attribute 'append'
```

```
# try to remove an element
fruits.remove('apple')
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-197-37543d45b6b4> in <module>
      1 # try to remove an element
----> 2 fruits.remove('apple')

AttributeError: 'tuple' object has no attribute 'remove'
```

You can also skip the parantheses (and) while creating a tuple. Python automatically converts comma-separated values into a tuple.

```
the_3_musketeers = 'Athos', 'Porthos', 'Aramis'
```

```
the_3_musketeers
```

```
('Athos', 'Porthos', 'Aramis')
```

You can also create a tuple with just one element by typing a comma after it. Just wrapping it with parentheses (and) won't make it a tuple.

```
single_element_tuple = 4,
```

```
single_element_tuple
```

```
(4,)
```

```
another_single_element_tuple = (4,)
```

```
another_single_element_tuple
```

```
(4,)
```

```
not_a_tuple = (4)
```

```
not_a_tuple
```

```
4
```

Tuples are often used to create multiple variables with a single statement.

```
point = (3, 4)
```

```
point_x, point_y = point
```

```
point_x
```

```
3
```

```
point_y
```

```
4
```

You can convert a list into a tuple using the `tuple` function, and vice versa using the `list` function

```
tuple(['one', 'two', 'three'])
```

```
('one', 'two', 'three')
```

```
list(('Athos', 'Porthos', 'Aramis'))
```

```
['Athos', 'Porthos', 'Aramis']
```

Tuples have just two built-in methods: `count` and `index`. Can you figure out what they do? While you look could look for documentation and examples online, there's an easier way to check a method's documentation, using the `help` function.

```
a_tuple = 23, "hello", False, None, 23, 37, "hello"
```

```
help(a_tuple.count)
```

```
Help on built-in function count:

    count(value, /) method of builtins.tuple instance
        Return number of occurrences of value.
```

Within a Jupyter notebook, you can also start a code cell with `?` and type the name of a function or method. When you execute this cell, you will see the function/method's documentation in a pop-up window.

```
?a_tuple.index
```

Try using `count` and `index` with `a_tuple` in the code cells below.

Start coding or generate with AI.

Start coding or generate with AI.

Dictionary

A dictionary is an unordered collection of items. Each item stored in a dictionary has a key and value. You can use a key to retrieve the corresponding value from the dictionary. Dictionaries have the type `dict`.

Dictionaries are often used to store many pieces of information e.g. details about a person, in a single variable. Dictionaries are created by enclosing key-value pairs within braces or curly brackets `{ and }`.

```
person1 = {
  'name': 'John Doe',
  'sex': 'Male',
```

```
    'age': 32,
    'married': True
}
```

```
person1
```

```
{'name': 'John Doe', 'sex': 'Male', 'age': 32, 'married': True}
```

Dictionaries can also be created using the `dict` function.

```
person2 = dict(name='Jane Judy', sex='Female', age=28, married=False)
```

```
person2
```

```
{'name': 'Jane Judy', 'sex': 'Female', 'age': 28, 'married': False}
```

```
type(person1)
```

```
dict
```

Keys can be used to access values using square brackets `[` and `]`.

```
person1['name']
```

```
'John Doe'
```

```
person1['married']
```

```
True
```

```
person2['name']
```

```
'Jane Judy'
```

If a key isn't present in the dictionary, then a `KeyError` is *thrown*.

```
person1['address']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-223-f2b8bd2476d4> in <module>
----> 1 person1['address']

KeyError: 'address'
```

You can also use the `get` method to access the value associated with a key.

```
person2.get("name")
```

```
'Jane Judy'
```

The `get` method also accepts a default value, returned if the key is not present in the dictionary.

```
person2.get("address", "Unknown")
```

```
'Unknown'
```

You can check whether a key is present in a dictionary using the `in` operator.

```
'name' in person1
```

```
True
```

```
'address' in person1
```

```
False
```

You can change the value associated with a key using the assignment operator.

```
person2['married']
```

```
False
```

```
person2['married'] = True
```

```
person2['married']
```

```
True
```

The assignment operator can also be used to add new key-value pairs to the dictionary.

```
person1
```

```
{'name': 'John Doe', 'sex': 'Male', 'age': 32, 'married': True}
```

```
person1['address'] = '1, Penny Lane'
```

```
person1
```

```
{'name': 'John Doe',
 'sex': 'Male',
 'age': 32,
 'married': True,
 'address': '1, Penny Lane'}
```

To remove a key and the associated value from a dictionary, use the `pop` method.

```
person1.pop('address')
```

```
'1, Penny Lane'
```

```
person1
```

```
{'name': 'John Doe', 'sex': 'Male', 'age': 32, 'married': True}
```

Dictionaries also provide methods to view the list of keys, values, or key-value pairs inside it.

```
person1.keys()
```

```
dict_keys(['name', 'sex', 'age', 'married'])
```

```
person1.values()
```

```
dict_values(['John Doe', 'Male', 32, True])
```

```
person1.items()
```

```
dict_items([('name', 'John Doe'), ('sex', 'Male'), ('age', 32), ('married',
True)])
```

```
person1.items()[1]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-239-b9fb361f2a9e> in <module>
----> 1 person1.items()[1]

TypeError: 'dict_items' object is not subscriptable
```

The results of `keys`, `values`, and `items` look like lists. However, they don't support the indexing operator `[]` for retrieving elements.

Can you figure out how to access an element at a specific index from these results? Try it below. *Hint: Use the `list` function*

Start coding or generate with AI.

Start coding or generate with AI.

Dictionaries provide many other methods. You can learn more about them here: https://www.w3schools.com/python/python_ref_dictionary.asp .

Here are some experiments you can try out with dictionaries (use the empty cells below):

- What happens if you use the same key multiple times while creating a dictionary?
- How can you create a copy of a dictionary (modifying the copy should not change the original)?
- Can the value associated with a key itself be a dictionary?
- How can you add the key-value pairs from one dictionary into another dictionary? Hint: See the `update` method.
- Can the dictionary's keys be something other than a string, e.g., a number, boolean, list, etc.?

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Further Reading

We've now completed our exploration of variables and common data types in Python. Following are some resources to learn more about data types in Python:

- Python official documentation: https://docs.python.org/3/tutorial/index.html
- Python Tutorial at W3Schools: https://www.w3schools.com/python/
- Practical Python Programming: https://dabeaz-course.github.io/practical-python/Notes/Contents.html

You are now ready to move on to the next tutorial: Branching using conditional statements and loops in Python

Let's save a snapshot of our notebook one final time using `jovian.commit` .

```
jovian.commit()
```

```
[jovian] Attempting to save notebook..
```

Questions for Revision

Try answering the following questions to test your understanding of the topics covered in this notebook:

1. What is a variable in Python?
2. How do you create a variable?
3. How do you check the value within a variable?
4. How do you create multiple variables in a single statement?
5. How do you create multiple variables with the same value?
6. How do you change the value of a variable?
7. How do you reassign a variable by modifying the previous value?
8. What does the statement `counter += 4` do?
9. What are the rules for naming a variable?
10. Are variable names case-sensitive? Do `a_variable`, `A_Variable`, and `A_VARIABLE` represent the same variable or different ones?
11. What is Syntax? Why is it important?
12. What happens if you execute a statement with invalid syntax?
13. How do you check the data type of a variable?
14. What are the built-in data types in Python?
15. What is a primitive data type?

16. What are the primitive data types available in Python?
17. What is a data structure or container data type?
18. What are the container types available in Python?
19. What kind of data does the Integer data type represent?
20. What are the numerical limits of the integer data type?
21. What kind of data does the float data type represent?
22. How does Python decide if a given number is a float or an integer?
23. How can you create a variable which stores a whole number, e.g., 4 but has the float data type?
24. How do you create floats representing very large (e.g., 6.023 x 10^23) or very small numbers (0.000000123)?
25. What does the expression `23e-12` represent?
26. Can floats be used to store numbers with unlimited precision?
27. What are the differences between integers and floats?
28. How do you convert an integer to a float?
29. How do you convert a float to an integer?
30. What is the result obtained when you convert 1.99 to an integer?
31. What are the data types of the results of the division operators `/` and `//`?
32. What kind of data does the Boolean data type represent?
33. Which types of Python operators return booleans as a result?
34. What happens if you try to use a boolean in arithmetic operation?
35. How can any value in Python be covered to a boolean?
36. What are truthy and falsy values?
37. What are the values in Python that evaluate to False?
38. Give some examples of values that evaluate to True.
39. What kind of data does the None data type represent?
40. What is the purpose of None?
41. What kind of data does the String data type represent?
42. What are the different ways of creating strings in Python?
43. What is the difference between strings creating using single quotes, i.e. `'` and `'` vs. those created using double quotes, i.e. `"` and `"`?
44. How do you create multi-line strings in Python?
45. What is the newline character, `\n`?
46. What are escaped characters? How are they useful?
47. How do you check the length of a string?
48. How do you convert a string into a list of characters?
49. How do you access a specific character from a string?
50. How do you access a range of characters from a string?

51. How do you check if a specific character occurs in a string?
52. How do you check if a smaller string occurs within a bigger string?
53. How do you join two or more strings?
54. What are "methods" in Python? How are they different from functions?
55. What do the `.lower`, `.upper` and `.capitalize` methods on strings do?
56. How do you replace a specific part of a string with something else?
57. How do you split the string "Sun,Mon,Tue,Wed,Thu,Fri,Sat" into a list of days?
58. How do you remove whitespace from the beginning and end of a string?
59. What is the string `.format` method used for? Can you give an example?
60. What are the benefits of using the `.format` method instead of string concatenation?
61. How do you convert a value of another type to a string?
62. How do you check if two strings have the same value?
63. Where can you find the list of all the methods supported by strings?
64. What is a list in Python?
65. How do you create a list?
66. Can a Python list contain values of different data types?
67. Can a list contain another list as an element within it?
68. Can you create a list without any values?
69. How do you check the length of a list in Python?
70. How do you retrieve a value from a list?
71. What is the smallest and largest index you can use to access elements from a list containing five elements?
72. What happens if you try to access an index equal to or larger than the size of a list?
73. What happens if you try to access a negative index within a list?
74. How do you access a range of elements from a list?
75. How many elements does the list returned by the expression `a_list[2:5]` contain?
76. What do the ranges `a_list[:2]` and `a_list[2:]` represent?
77. How do you change the item stored at a specific index within a list?
78. How do you insert a new item at the beginning, middle, or end of a list?
79. How do you remove an item from al list?
80. How do you remove the item at a given index from a list?
81. How do you check if a list contains a value?
82. How do you combine two or most lists to create a larger list?
83. How do you create a copy of a list?
84. Does the expression `a_new_list = a_list` create a copy of the list `a_list`?
85. Where can you find the list of all the methods supported by lists?
86. What is a Tuple in Python?
87. How is a tuple different from a list?

88. Can you add or remove elements in a tuple?
89. How do you create a tuple with just one element?
90. How do you convert a tuple to a list and vice versa?
91. What are the count and index method of a Tuple used for?